TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN

ĐỀ KIỂM TRA GIỮA KỲ

Tên học phần: Toán ứng dụng CNTT

Mã học phần: 1023220 Số tín chỉ: 03

Phương pháp đánh giá (*): Tự luận Thời gian làm bài: 90 phút

Đề số: 06

☐ Sinh viên không được sử dụng tài liệu khi làm bài.

R Sinh viên được sử dụng tài liệu khi làm bài.

Họ tên: Đàm Phú Quý **Lớp**: 24T_DT2 **MSSV**: 102240107

Sinh viên làm bài trực tiếp trên tệp này, lưu tệp với định dạng MSSV_HọTên.pdf và nộp bài thông qua MSTeam

Câu 1 (***3 điểm,*** đáp ứng CLO1, CLO2, CLO3, CLO4): Cho số nguyên dương N= 8799. Viết chương trình bằng C/C++ có sử dụng hàm thực hiện:

- Phân tích thừa số nguyên tố đối với số N
- Tìm số M là ước số nguyên tố lớn nhất của N
- Liệt kê các số nguyên tố nhỏ hơn M và tính tổng của chúng
- Liệt kê các số hoàn hảo nhỏ hơn M và tính tổng của chúng.

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
#include <vector>

using namespace std;

bool isPrime(long long n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
```

```
    return true;
}

vector<pair<long long, int>> primeFactors(long long n) {
    vector<pair<long long, int>> factors;
    if (n % 2 == 0) {
        int count = 0;
        while (n % 2 == 0) {
            count++;
            n /= 2;
        }
        factors.push_back({2, count});
    }
    for (long long i = 3; i * i <= n; i += 2) {
        if (n % i == 0) {
            int count = 0;
            while (n % i == 0) {
                count++;
                n /= i;
            }
            factors.push_back({i, count});
        }
    }
    if (n > 1) {
        factors.push_back({n, 1});
    }
    return factors;
}

long long maxPrimeFactor(long long n) {
```

```
    vector<pair<long long, int>> factors = primeFactors(n);
    if (factors.empty()) return -1;
    return factors.back().first;
}

bool isPerfect(long long n) {
    if (n <= 1) return false;
    long long sum = 1;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            sum += i;
            if (i * i != n) {
                sum += n / i;
            }
        }
    }
    return sum == n;
}

pair<vector<long long>, long long> primeNumbersLessThanAndSum(long long m) {
    vector<long long> primes;
    long long sum = 0;
    for (long long i = 2; i < m; i++) {
        if (isPrime(i)) {
            primes.push_back(i);
            sum += i;
        }
    }
    return {primes, sum};
}
```

```
pair<vector<long long>, long long> perfectNumbersLessThanAndSum(long long m) {
    vector<long long> perfectNumbers;
    long long sum = 0;
    for (long long i = 2; i < m; i++) {
        if (isPerfect(i)) {
            perfectNumbers.push_back(i);
            sum += i;
        }
    }
    return {perfectNumbers, sum};
}

int main() {
    long long n;
    cout << "Nhap so nguyen duong n: ";
    cin >> n;

    vector<pair<long long, int>> factors = primeFactors(n);
    cout << "Cac thua so nguyen to cua " << n << " la: ";
    for (size_t i = 0; i < factors.size(); i++) {
        cout << factors[i].first << "^" << factors[i].second;
        if (i + 1 < factors.size()) {
            cout << " * ";
        }
    }
    cout << endl;

    long long m = maxPrimeFactor(n);
    cout << "Uoc so nguyen to lon nhat cua " << n << " la: " << m << endl;

    pair<vector<long long>, long long> primeResult = primeNumbersLessThanAndSum(m);
```

```
    cout << "Cac so nguyen to nho hon " << m << " la: ";
    for (long long prime : primeResult.first) {
        cout << prime << " ";
    }
    cout << endl;
    cout << "Tong cua cac so nguyen to nho hon " << m << " la: " << primeResult.second << endl;

    pair<vector<long long>, long long> perfectResult = perfectNumbersLessThanAndSum(m);
    cout << "Cac so hoan hao nho hon " << m << " la: ";
    for (long long perfect : perfectResult.first) {
        cout << perfect << " ";
    }
    cout << endl;
    cout << "Tong cua cac so hoan hao nho hon " << m << " la: " << perfectResult.second << endl;

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới :

```
Nhap so nguyen duong n: 8799
Cac thua so nguyen to cua 8799 la: 3^1 * 7^1 * 419^1
Uoc so nguyen to lon nhat cua 8799 la: 419
Cac so nguyen to nho hon 419 la: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109 113 127 131 137 139 149 151 157 163 167 173
1 193 197 199 211 223 227 229 233 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331 337 347 349 353 359 367 373 379 383 389 397 401 409
Tong cua cac so nguyen to nho hon 419 la: 14697
Cac so hoan hao nho hon 419 la: 6 28
Tong cua cac so hoan hao nho hon 419 la: 34
```

***Câu 2*: (2 điểm,** đáp ứng CLO1, CLO2, CLO3, CLO4*) Cho hệ phương trình đồng dư sau*

$$\begin{cases} x \equiv 1 \bmod 3 \\ x \equiv 4 \bmod 5 \\ x \equiv 6 \bmod 8 \\ x \equiv 9 \bmod 11 \\ x \equiv 13 \bmod 19 \end{cases}$$

- Viết chương trình C/C++ có sử dụng hàm giải hệ phương trình đồng dư trên.

Trả lời: Dán code vào bên dưới:

```
#include <iostream>
using namespace std;

// extended Euclidean algorithm
long long extendedGCD(long long a, long long b,
                      long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }

    long long x1, y1;
    long long gcd = extendedGCD(b, a % b, x1, y1);

    x = y1;
    y = x1 - (a / b) * y1;

    return gcd;
}

// modulo inverse
long long modInverse(long long a, long long m) {
    long long x, y;
    extendedGCD(a, m, x, y);
    return (x % m + m) % m;
}

// Chinese Remainder Theorem
long long CRT(long long a[], long long m[], int n) {
    long long M = 1;
```

```
    for (int i = 0; i < n; i++)
        M *= m[i];

    long long x = 0;

    for (int i = 0; i < n; i++) {
        long long Mi = M / m[i];
        long long yi = modInverse(Mi, m[i]);

        x += a[i] * Mi * yi;
    }

    return (x % M + M) % M;
}

int main() {
    int n;
    cout << "Nhap so luong phuong trinh dong du: ";
    cin >> n;

    long long a[n], m[n];

    cout << "Nhap cac he so va moduli: " << endl;
    for (int i = 0; i < n; i++) {
        cout << i + 1 << ": ";
        cin >> a[i] >> m[i];
    }

    long long x = CRT(a, m, n);
```

```
    long long M = 1;
    for (int i = 0; i < n; i++)
        M *= m[i];

    cout << x << " (mod " << M << ")" << endl;

    cout << "Dang tong quat: x = " << x << " + k * " << M << ", voi k la mot so nguyen" << endl;

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới:

```
Nhap so luong phuong trinh dong du: 5
Nhap cac he so va moduli:
1: 1 3
2: 4 5
3: 6 8
4: 9 11
5: 13 19
15574 (mod 25080)
Dang tong quat: x = 15574 + k * 25080, voi k la mot so nguyen
```

Câu 3: (**5** ***điểm,*** đáp ứng CLO1, CLO2, CLO3, CLO4) Cho ma trận

$$A = \begin{bmatrix} 5 & 4 & 8 \\ 4 & 9 & 7 \\ 8 & 7 & 12 \end{bmatrix}.$$

Viết chương trình bằng c/c++ có sử dụng hàm thực hiện phân rã ma trận A (có hàm kiểm tra điều kiện phân rã).

a) Phân rã **Cholesky** LDL^T ma trận A

Trả lời: Dán code vào bên dưới (bao gồm điều kiện của ma trận A nếu có):

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
```

```
#include <algorithm>

using namespace std;

const double EPS = 1e-5;

void printMatrix(const vector<vector<double>>& A, int precision = 6) {
    int rows = A.size();
    if (rows == 0) return;
    int cols = A[0].size();

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            double val = A[i][j];
            if (fabs(val) < EPS) val = 0.0;
            cout << setw(13) << fixed << setprecision(precision) << val;
        }
        cout << '\n';
    }
}

void printVector(const vector<double>& v, int precision = 6) {
    for (size_t i = 0; i < v.size(); i++) {
        double val = v[i];
        if (fabs(val) < EPS) val = 0.0;
        cout << setw(13) << fixed << setprecision(precision) << val;
    }
    cout << '\n';
}

bool isSymmetric(const vector<vector<double>>& A) {
```

```
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (fabs(A[i][j] - A[j][i]) > EPS) {
                return false;
            }
        }
    }
    return true;
}

double determinant(vector<vector<double>> M) {
    int n = M.size();
    double det = 1.0;

    for (int i = 0; i < n; i++) {
        int pivot = i;
        for (int row = i + 1; row < n; row++) {
            if (fabs(M[row][i]) > fabs(M[pivot][i])) {
                pivot = row;
            }
        }

        if (fabs(M[pivot][i]) < EPS) {
            return 0.0;
        }

        if (pivot != i) {
            swap(M[i], M[pivot]);
            det = -det;
        }
```

```
        det *= M[i][i];

        for (int row = i + 1; row < n; row++) {
            double factor = M[row][i] / M[i][i];
            for (int col = i + 1; col < n; col++) {
                M[row][col] -= factor * M[i][col];
            }
        }
    }

    return det;
}

vector<vector<double>> getLeadingPrincipalSubmatrix(const vector<vector<double>>& A, int
k) {
    vector<vector<double>> sub(k, vector<double>(k));
    for (int i = 0; i < k; i++) {
        for (int j = 0; j < k; j++) {
            sub[i][j] = A[i][j];
        }
    }
    return sub;
}

bool canLDLT(const vector<vector<double>>& A, vector<double>& minors) {
    int n = A.size();
    minors.clear();

    if (!isSymmetric(A)) {
        cout << "-> Vi pham: Ma tran A khong doi xung (A != A^T).\n";
```

```
        return false;
    }

    bool isPositiveDefinite = true;
    for (int k = 1; k <= n; k++) {
        vector<vector<double>> sub = getLeadingPrincipalSubmatrix(A, k);
        double detK = determinant(sub);
        minors.push_back(detK);

        if (detK <= EPS) {
            isPositiveDefinite = false;
        }
    }

    return isPositiveDefinite;
}

void LDLT(const vector<vector<double>>& A,
          vector<vector<double>>& L,
          vector<double>& D) {
    int n = A.size();
    L.assign(n, vector<double>(n, 0.0));
    D.assign(n, 0.0);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= i; j++) {
            double sum = 0.0;
            for (int k = 0; k < j; k++) {
                sum += L[i][k] * D[k] * L[j][k];
            }
```

```
            if (i == j) {
                D[i] = A[i][i] - sum;
                L[i][j] = 1.0;
            } else {
                L[i][j] = (A[i][j] - sum) / D[j];
            }
        }
    }
}

int main() {
int n;
    cout << "Nhap cap cua ma tran vuong n: ";
    if (!(cin >> n) || n <= 0) {
        cerr << "Kich thuoc ma tran khong hop le!\n";
        return 1;
    }

    vector<vector<double>> A(n, vector<double>(n));
    cout << "Nhap cac phan tu cua ma tran A (" << n << "x" << n << "):\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> A[i][j];
        }
    }

    cout << "MA TRAN A BAN DAU (" << n << "x" << n << "):\n";
    printMatrix(A);

    cout << "\nPHAN RA LDL^T:\n";
    vector<double> minors;
```

```
    bool okLDLT = canLDLT(A, minors);

    if (minors.size() == (size_t)n) {
        cout << "Cac dinh thuc con chinh dan dau:\n";
        for (int k = 1; k <= n; k++) {
            cout << "  Delta_" << k << " = " << fixed << setprecision(6) << minors[k - 1];
            if (minors[k - 1] > EPS) {
                cout << " > 0\n";
            } else {
                cout << " <= 0 (Khong xac dinh duong)\n";
            }
        }
    }

    if (okLDLT) {
        cout << "Ma tran A doi xung va xac dinh duong. Thoa man dieu kien phan ra
LDL^T.\n\n";

        vector<vector<double>> L;
        vector<double> D;
        LDLT(A, L, D);

        cout << "Ma tran tam giac duoi L (duong cheo = 1):\n";
        printMatrix(L);

        cout << "\nVector duong cheo D:\n";
        printVector(D);

        cout << "\nMa tran duong cheo D dang day du:\n";
        vector<vector<double>> D_matrix(n, vector<double>(n, 0.0));
        for (int i = 0; i < n; i++) D_matrix[i][i] = D[i];
```

```
        printMatrix(D_matrix);
    } else {
        cout << "Khong the phan ra LDL^T vi ma tran A khong thoa man dieu kien (khong doi xung hoac khong xac dinh duong).\n";
    }
}
```

Trả lời: Dán kết quả thực thi vào bên dưới (sai số $e = 10^{-5}$):

```
Nhap cac phan tu cua ma tran A (3x3):
15 4 8
4 9 7
8 7 12
MA TRAN A BAN DAU (3x3):
    15.000000     4.000000     8.000000
     4.000000     9.000000     7.000000
     8.000000     7.000000    12.000000

PHAN RA LDL^T:
Cac dinh thuc con chinh dan dau:
  Delta_1 = 15.000000 > 0
  Delta_2 = 119.000000 > 0
  Delta_3 = 565.000000 > 0
Ma tran A doi xung va xac dinh duong. Thoa man dieu kien phan ra LDL^T.

Ma tran tam giac duoi L (duong cheo = 1):
     1.000000     0.000000     0.000000
     0.266667     1.000000     0.000000
     0.533333     0.613445     1.000000

Vector duong cheo D:
    15.000000     7.933333     4.747899

Ma tran duong cheo D dang day du:
    15.000000     0.000000     0.000000
     0.000000     7.933333     0.000000
     0.000000     0.000000     4.747899
```

b) Phân rã **eigendecomposition** ma trận A

Trả lời: Dán code vào bên dưới (bao gồm điều kiện của ma trận A nếu có):

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <algorithm>

using namespace std;

const double EPS = 1e-5;

void printMatrix(const vector<vector<double>>& A, int precision = 6) {
    int rows = A.size();
    if (rows == 0) return;
    int cols = A[0].size();

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            double val = A[i][j];
            if (fabs(val) < EPS) val = 0.0; // Tránh hiển thị -0.000000 theo sai số e = 1e-5
            cout << setw(13) << fixed << setprecision(precision) << val;
        }
        cout << '\n';
    }
}

void printVector(const vector<double>& v, int precision = 6) {
    for (size_t i = 0; i < v.size(); i++) {
        double val = v[i];
```

```
        if (fabs(val) < EPS) val = 0.0;
        cout << setw(13) << fixed << setprecision(precision) << val;
    }
    cout << '\n';
}

bool isSymmetric(const vector<vector<double>>& A) {
    int n = A.size();
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (fabs(A[i][j] - A[j][i]) > EPS) {
                return false;
            }
        }
    }
    return true;
}

bool canEigen(const vector<vector<double>>& A) {
    return isSymmetric(A);
}

void eigenDecomposition(vector<vector<double>> A,
                        vector<vector<double>>& Q,
                        vector<double>& lambda,
                        double eps) {
    int n = A.size();


    Q.assign(n, vector<double>(n, 0.0));
    for (int i = 0; i < n; i++) {
```

```
    Q[i][i] = 1.0;
}

if (n <= 1) {
    lambda.resize(n);
    if (n == 1) lambda[0] = A[0][0];
    return;
}

int maxIterations = 10000;
int iter = 0;

while (iter++ < maxIterations) {
    int p = 0, q = 1;
    double maxVal = fabs(A[0][1]);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (fabs(A[i][j]) > maxVal) {
                maxVal = fabs(A[i][j]);
                p = i;
                q = j;
            }
        }
    }

    if (maxVal < eps) {
        break;
    }

    double theta = 0.5 * atan2(2.0 * A[p][q], A[p][p] - A[q][q]);
```

```
    double c = cos(theta);
    double s = sin(theta);

    double App = A[p][p];
    double Aqq = A[q][q];
    double Apq = A[p][q];

    A[p][p] = c * c * App + 2.0 * c * s * Apq + s * s * Aqq;
    A[q][q] = s * s * App - 2.0 * c * s * Apq + c * c * Aqq;
    A[p][q] = 0.0;
    A[q][p] = 0.0;

    for (int k = 0; k < n; k++) {
        if (k != p && k != q) {
            double Akp = A[k][p];
            double Akq = A[k][q];

            A[k][p] = A[p][k] = c * Akp + s * Akq;
            A[k][q] = A[q][k] = -s * Akp + c * Akq;
        }
    }

    for (int k = 0; k < n; k++) {
        double Qkp = Q[k][p];
        double Qkq = Q[k][q];

        Q[k][p] = c * Qkp + s * Qkq;
        Q[k][q] = -s * Qkp + c * Qkq;
    }
}
```

```
    lambda.resize(n);
    for (int i = 0; i < n; i++) {
        lambda[i] = A[i][i];
    }
}

int main() {
    int n;
    cout << "Nhap cap cua ma tran vuong n: ";
    if (!(cin >> n) || n <= 0) {
        cerr << "Kich thuoc ma tran khong hop le!\n";
        return 1;
    }

    vector<vector<double>> A(n, vector<double>(n));
    cout << "Nhap cac phan tu cua ma tran A (" << n << "x" << n << "):\n";
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cin >> A[i][j];
        }
    }
    cout << "\nPhan eigendecomposition:\n";
    cout << "Kiem tra dieu kien phan ra\n";

    if (canEigen(A)) {
        cout << "Ma tran A doi xung thuc. Thoa man dieu kien Jacobi.\n\n";

        vector<vector<double>> Q;
        vector<double> lambda;
        eigenDecomposition(A, Q, lambda, EPS);
```

```
        cout << "Cac tri rieng:\n";
        printVector(lambda);

        cout << "\nMa tran vector rieng Q (cac cot la cac vector rieng truc chuan tuong ung):\n";
        printMatrix(Q);

        cout << "\nKiem tra Lambda dang ma tran duong cheo:\n";
        vector<vector<double>> Lambda_matrix(n, vector<double>(n, 0.0));
        for (int i = 0; i < n; i++) Lambda_matrix[i][i] = lambda[i];
        printMatrix(Lambda_matrix);
    } else {
        cout << "Khong the ap dung phuong phap Jacobi vi ma tran A khong doi xung thuc.\n";
    }

    return 0;
}
```

Trả lời: Dán kết quả thực thi vào bên dưới (sai số $e = 10^{-5}$):

```
Nhap cap cua ma tran vuong n: 3
Nhap cac phan tu cua ma tran A (3x3):
15 4 8
4 9 7
8 7 12

Phan eigendecomposition:
Kiem tra dieu kien phan ra
Ma tran A doi xung thuc. Thoa man dieu kien Jacobi.

Cac tri rieng:
    25.237885     7.944025     2.818090

Ma tran vector rieng Q (cac cot la cac vector rieng truc chuan tuong ung):
     0.654235    -0.710237    -0.259883
     0.429512     0.631762    -0.645287
     0.622491     0.310547     0.718377

Kiem tra Lambda dang ma tran duong cheo:
    25.237885     0.000000     0.000000
     0.000000     7.944025     0.000000
     0.000000     0.000000     2.818090
```